

AI Tutoring System - "北辰" Běichén

"He who governs by virtue is like the North Star, which remains in its place while all the other stars revolve around it." – Confucius

The objective of this project is to build a prototype AI tutoring system that uses agentic design to help a student learn a specific topic.

Key functionalities/tasks may include:

1. **Tutoring Strategy Design:** Select one specific topic to tutor and one tutoring strategy (e.g., Socratic questioning, scaffolded hinting with fading, or inquiry-based guidance). Define the pedagogical rules the AI must follow.
 2. **Tutoring Chatbot Implementation:** Build a chat interface connected to an LLM API. Engineer the system prompt to enforce the chosen tutoring strategy. Implement basic state tracking so the AI remembers what the learner has attempted.
 3. **Learner Testing and Iteration:** Test the tutor with real learners (other group members or volunteers). Observe where the AI violates its pedagogical rules, gives away answers prematurely, or provides confusing guidance. Iterate on the system prompt and interaction design based on test results.
 4. **Tutoring Quality Reflection:** Analyze what the AI Tutor did well and where it failed from a pedagogical perspective. Discuss how explicit tutoring strategies and constraints can improve LLM-based educational systems, and what limitations remain.
-

Our Chosen Topic

Geography

Specifically, Physical Geography – we consider it an ideal topic for proof-of-concept for 2 reasons:

- I. Geographical Phenomena have a **clear Cause & Effect**, so the inquiry scope isn't too intensive for the system.
 - II. Physical Geography utilises **clear Terminology & Synonyms**, therefore it will not be taxing for the system to equate nomenclature.
-

Our Tutoring Strategy

Inquiry-Based Learning

Adopting a fusion of Pedaste’s ‘5-Phase’ model as the process architecture and Bybee’s ‘5 Es’ model for the “move” taxonomy.

However, LLMs are optimised to answer questions directly. Inquiry-Based Learning demands the opposite, defined by the goal to guide learners towards constructing understanding themselves.

Our system therefore works *against* the model’s default optimisation, constraining the LLM through a structured pedagogical state machine that regulates when and how it may ask questions, request evidence, scaffold learning, or provide increasingly explicit guidance. Each question is transposed into a conceptual dependency graph, then that graph is traversed pedagogically, *literally mapping and guiding the Learner from point A (nescience) to point B (science).*

The underlying design philosophy is that **the LLM should not decide the pedagogy**. The *architecture* decides what Phase the Learner is in, what Component they are on, whether evidence is needed, whether to hint, and whether to conclude; the LLM is merely generating the language within those constraints.

The novelty, then, lies less in withholding answers (a known Socratic technique) than in the **dependency-graph decomposition and state enforcement of inquiry over a model otherwise optimised to short-circuit it**.

A secondary contribution is adapting two frameworks designed for human classrooms to a 1:1 text dialogue between a human “Learner” and a computational “Tutor”, documenting specifically what we change and why.

Pedagogical Rules for the AI

Primarily, the AI Tutor must never state the answer to the Origin Query; it has an Internal Answer (*ANS*), however that is purely for backend functionality and testing, it is not to be disclosed overtly.

The AI Tutor must motivate and encourage the Learner where needed, but also remain level-headed to retain a professional Tutor-Student dynamic and an air of trustworthiness and reliability.

Per Turn, only 1 Phase (see “Phases” below) may be occupied.

Exhaustion Fallback: If the Learner cannot progress after 3 attempted Turns on a specific Phase, trigger the Fallback Condition, which shall do one of the following dependent on what is most appropriate contextually:

- A. Reframe/Narrow the Inquiry proposed by the AI Tutor
 - B. Provide the Learner with a Hint, then allow them to reattempt answering the Inquiry
 - C. Partially Scaffold, allowing the Learner to complete the intended resolution of the Inquiry
 - D. If options A, B and C are exhausted, the AI Tutor is permitted to provide a near-direct prompt if not the answer to the Inquiry within that Phase
-

Question Analysis & Conclusion/Assertion Decomposition

Two key aspects that dictate the potency of the system are the capability and methodology behind the Analysis of the Learner's Origin Question alongside the Decomposition of the Target Conclusion as to resolve an optimal path to understanding. The processes will be outlined below.

I. Question Analysis

e.g "Why do clouds exist?"

Two things would be done immediately. Keywords would be derived from the Learner's Origin Question under two brackets: Content and Grammar.

Content Keywords answer to the system what the Question is about; operative terminology needs to be queried to the Grounded Knowledge Base (*in this case, Wikipedia API would be queried about Clouds*) and verbs/actions help shape the intention (*in this case, "exist" defines that the Question is about formation and persistence, which would narrow the system to focus on Cloud Formation*).

Grammatical Keywords dictate to the system the shape of the answer:

- "Why" = causal chain
- "How" = process / sequence
- "When / Where" = conditional / contextual

These are very important for the template used for the Conclusion Decomposition. Compound questions (carrying more than one interrogative) are decomposed against multiple templates and reconciled at the Decomposition stage (see below).

We would then create something as following:

```
{  
  operative: "clouds",  
  action: "exist",  
  question_type: "why",
```

```
target_conclusion: "...",
components: [ ... ]
}
```

`target_conclusion` is filled by a distinct step, run in the Orientation Phase: the Origin Question is passed against the Grounded Knowledge Base (*Wikipedia API*) to produce an internal Conclusion validated against retrieved material — not generated from the model's unverified memory. That grounded Conclusion is what Section II below decomposes.

II. Conclusion Decomposition

e.g. “water vapour evaporates, cools, then condenses...”

This would be updated within `target_conclusion`, then the terminology will be abstracted. The Terminology is vocabulary the Learner must be familiarised with, acting as nodes. From these terms, relative to the original Question, are the Premises/Concepts, the propositions the Learner must understand. These act as edges.

A Component is the smallest conceptual unit whose understanding is necessary to justify the Target Conclusion.

Critically, a Component stores *both* the node and the edge: the `concept` (*the node term, e.g. “adiabatic cooling”*) and the `statement` (*the causal claim itself in plain text, e.g. “rising air cools because pressure drops”*); without the latter, the system would be tracking scores about a proposition it never stored.

The Components are ranked (*depending on the template yielded by the auxiliary term*) to optimise the AI Tutor's Inquiry flow to guide the Learner to the Conclusion with the most ease. The Components are then fed into the state machine to populate the `required_components` list.

Compound interrogatives do not require a separate hierarchy system. Both frames are decomposed into one shared Component set and ranked together by prerequisite dependency; where one frame's Components depend on the other's, the ordering sequences them automatically — typically mechanism before consequence, but determined per Question rather than fixed in advance. Where the frames are genuinely independent, one is pursued as the main Inquiry and the other is parked in `sub_inquiries` for Phase V.

Phases

As previously stated, we are using a fusion of the 5-Phase & 5 E models in the following capacity:

I. Orientation (Engage)

The AI Tutor parses the Learner's Question, creates an internal Conclusion validated against grounding material, then breaks the Conclusion down into parts then ranks them in a progression the Learner can follow "from First Principles" so to speak.

For each Question proposed by the Learner, a **Session object** is created (*InquirySession*), holding the metadata for the entire Pedagogical Process. Note the distinction in naming: the **state machine** is the phase-transition logic (the FSM that moves Orientation → Conceptualisation → ...); the object below is the **session state** the state machine operates *on*. They are separate.

- *topic_anchor* – a stable reference to the Learner's Question.
- *target_conclusion* – the LLM's plaintext answer to the Question; what the conversation should steer towards.
- *required_components* – the abstracted Premises necessary to reach the *target_conclusion*. A list of Component objects (defined below); the *next* field on each Component turns the list into a Graph that may be mapped.
- *hypothesis* / *hypothesis_status* – the Learner's single driving Hypothesis at the level of the Origin Question, and its current classification (see Phase II).
- *evidence_repository* – Evidence already surfaced, so it is not unnecessarily repeated.
- *sub_inquiries* – parked tangents, to be explored in Phase V.
- *transcript* – the full turn log, consumed post-hoc by the Recursive Audit.

The number of Components to a Conclusion serves as a clear, linear approximator of a Question's complexity. The AI Tutor analyses this during Orientation; rather than mapping it onto an arbitrary numeric scale (which invites the model to fabricate a figure), **Component count serves as the primary approximation of complexity** — a coarse but legible proxy, not a precise measure (two Components can differ widely in difficulty), optionally bucketed (low / medium / high) where a coarser handle is useful.

Components are not a fixed state upon the system processing and decomposing the Origin Question; instead, they act more like a checklist and are mutable. The system:

- **Collapses** Components the Learner demonstrably grasps.
- **Appends** Components when the Learner's Hypothesis introduces unexpected, but valid, themes.

It is imperative to note that spawning new Components requires proper consideration for eligibility under the following question:

Does resolving this potential Component establish a premise the Conclusion needs/would benefit from?

- If Yes, Append & Investigate.
- If No, Park as a **sub_inquiry**, to be explored within Phase V.

Another reason Components are important to outline well is due to them determining the quality of the AI Tutor's Inquiries; the Inquiries must be as valuable, if not more, than the Learner's Hypotheses/Answers.

The Component object (single authoritative schema — coverage is *derived* from mastery and groundedness):

Component

— concept	# node term, e.g. "adiabatic cooling"
— statement	# the premise in plain text, e.g. "rising air cools because pressure drops"
— mastery	# 0–1: does the Learner appear to understand it
— groundedness	# 0–1: has that understanding been justified by reasoning/evidence
— attempts	# int
— evidence_used	# list of Evidence references
— next	# the dependency edge → next Component (graph traversal)
— confidence	# optional: Tutor's confidence in its mastery estimate

A Component is considered **covered** when both **mastery** and **groundedness** cross a threshold of declarative understanding — it is a computed status, not a stored flag.

Process flow per Component:

Current Component

- ↓ Assess mastery
- ↓ Need evidence?
- ↓ Need inquiry?
- ↓ Need hint?
- ↓ Covered? (mastery & groundedness ≥ threshold)
- ↓ Advance

The AI Tutor advances to the next Phase when the Learner is oriented and ready to begin answering their own Question with the Tutor's guidance. The Graph is initiated and designed to drive the entire process, rather than improvising pedagogy when LLMs are optimised for answering.

II. Conceptualisation (Explore) — *elicit & classify*

The AI Tutor elicits a **single driving Hypothesis** from the Learner at the level of the Origin Question, to derive their current model of the aspects pertinent to the Question, irrespective of objective fact.

(Micro-predictions about individual Components may be drawn out later where natural, but the Learner holds one central Hypothesis they are investigating — not one per Component; the latter would degrade inquiry into a worksheet.)

The role of this Phase is to **elicit and classify** — it sorts the Hypothesis into one of three cases. The *acting* upon each case happens in Phase III; the boundary is clean: Conceptualisation diagnoses, Investigation treats.

A. The Learner posits a False Hypothesis

The goal is to disconfirm without explicitly saying the Learner is incorrect. The AI Tutor should surface evidence that stress tests the position until the Learner sees the contradiction(s) on their own and revises their position. This arguably has the highest value, seeding key Critical Thinking skills.

B. The Learner posits a True Hypothesis, but it is Ungrounded

The goal is to ground the position with evidence and deeper understanding. The AI Tutor should — even though it's true — treat the Hypothesis as unproven until the Learner can decisively and robustly defend the position. Hence, they should guide and steer the Learner to reinforce their belief, in turn improving Retention and Confidence.

C. The Learner posits a True Hypothesis, which is Grounded

There is no goal in this case, besides identifying this as early as possible so that the AI Tutor may move onto the next Component.

In essence, this Phase is heavily dependent on asking guiding Inquiries that pull predictions from the Learner that allow the AI Tutor to have a more intimate understanding of the Learner's familiarity with the relevant Components to the Origin Question, in turn allowing the AI Tutor to optimally close the gap between the Learner and the Target Conclusion.

Adaptive Evidence Integration

Although evidence is central to Inquiry-Based Learning, requiring the Learner to justify every assertion with external evidence would be both pedagogically and practically counterproductive, deterring the Learner from the process as well as being overkill in many cases. Such an approach risks interrupting the natural flow of

inquiry, increasing cognitive load, and reducing learner engagement by turning the dialogue into a repetitive source-finding exercise rather than an investigation.

Instead, the AI Tutor treats evidence as an adaptive pedagogical instrument rather than a mandatory checkpoint. The Tutor continually evaluates whether introducing evidence would meaningfully progress the Learner's understanding within the current Component.

Evidence may therefore be requested under circumstances such as:

- The Learner presents a hypothesis that requires empirical support.
- Multiple competing hypotheses exist and must be distinguished.
- The Learner demonstrates uncertainty or low confidence.
- The Tutor identifies a persistent misconception requiring contradiction through reliable evidence.
- The current Component represents a foundational premise upon which subsequent Components depend.

It is worth noting that some Learners may benefit from strict adherence to empirical provision, something a verbose version of this system would naturally accommodate for.

Where the Learner is demonstrating productive reasoning and the Inquiry continues to progress naturally, the Tutor may defer evidence gathering in favour of continued questioning and conceptual exploration.

Accordingly, each Component maintains an internal **Groundedness** state, distinct from conceptual **Mastery**. Mastery reflects whether the Learner appears to understand a concept, whereas Groundedness reflects whether that understanding has been sufficiently justified through reasoning or supporting evidence. The Tutor balances these two measures when deciding whether to continue the Inquiry, introduce evidence, provide scaffolding, or progress to the next Component.

III. Investigation (Explore ↔ Evaluate) — *act on the classification*

This is the loop of Grounded Evidence and Knowledge Validation, and it is where the Tutor *acts* on whichever of the three cases Phase II diagnosed (*disconfirm the false, ground the true-ungrounded, or move past the true-grounded*).

During Investigation, the AI Tutor evaluates whether additional evidence would meaningfully improve the Learner's understanding of the current Component. Where appropriate, the Tutor may introduce evidence itself or encourage the Learner to locate and interpret supporting material. Evidence therefore serves to validate, challenge, or refine hypotheses rather than acting as a compulsory requirement for every conversational turn. The Learner is then asked to relate this to their Hypothesis in whatever way applies (*supporting / complicating / defying*).

Evidence Provided is tracked, within an “Evidence Repository” list {evidence (Evidence object), covered (Boolean)} to ensure it is not unnecessarily repeated.

IV. Conclusion (Explain → Evaluate)

Once all Required Components have been covered to a degree the AI Tutor deems suitable (when it believes the Learner has metabolised the information), it offers an “off-ramp”:

“I think we have enough for you to try to answer your question now — would you like to try?”

Note — this isn’t a forced path; if the Learner desires to yield more clarity they are the most reliable source on whether the information has been internalised. If the Learner does not wish to Conclude, it simply returns the process to the prior Phases after the AI Tutor inquires upon what needs more clarification.

Alternatively, if the Learner is ready to conclude, they state their answer. The AI Tutor may then confirm if it is akin to the Target Conclusion. If not, it analyses the Asserted Incorrect Answer, decomposing which component seems to be causing the misunderstanding then returns to the prior Phases, offering meaningful Inquiry to remedy the misnomers.

Termination guard: to prevent an indefinite loop, repeated failed Conclusions on the same Component escalate the scaffolding (see Exhaustion Fallback, rung D) rather than returning the Learner to the same Inquiry indefinitely.

If the Question is answered, the AI then enters the **macro-evaluation** (of the *Learner*). It offers 2 things:

1. **Pop Quiz** — a concise test on the components relevant to the question to validate the Learner grasps the topics covered.
2. **Survey** — the AI Tutor flips the script, asking the Learner a few questions so it may build a better profile of how the Learner thinks about things, what they find easy, what they find hard. This information is then stored in a persistent Learner Profile. Questions may include:
 - “Which parts of what we covered were hard to understand?”
 - “Which parts of what we covered were easy to understand?”
 - “Were any of my questions confusing?”

Note, down the line if the system is to be expanded, this could be an inter-hoc process, however that is beyond the required technical scope.

V. Discussion (Evaluate → Elaborate)

The AI Tutor now asks if any new Questions have stemmed off this dialogue. Alternatively, if any tangential questions were asked during the dialogue (parked in *sub_inquiries*), the AI Tutor may jump into explaining

those now. A new Question raised here loops back to Conceptualisation as a fresh cycle — this is what makes the process a cycle rather than a line.

Recursive Evaluation (Auditing the *Tutor*)

The macro-evaluation above assesses the *Learner*. Distinct from it, and central to Tasks 3 and 4, is the **Recursive Audit**: a second LLM pass that evaluates the *Tutor* against its own Pedagogical Rules.

It runs **post-hoc**, not in the live loop — it consumes the saved **transcript** after a session and flags, turn by turn, where the Tutor violated a rule (*gave away the answer prematurely, occupied more than one Phase in a turn, advanced a Phase before its exit condition, provided confusing guidance*), tagging each flag with the specific rule breached. It is genuinely *recursive* in that the same model architecture audits its own output, and it requires no additional live infrastructure.

This is the artifact that makes the project assessable: *Task 3* (“observe where the AI violates its rules”) and *Task 4* (“analyse what it did well and where it failed”) are both fed directly by the audit log rather than by recollection after the fact. The thinking/reasoning mode of the model is used for this pass, where a reasoning trace measurably improves the quality of the adherence judgement.

Balancing Guidance & Autonomy

“Effective Inquiry-Based education requires a balance between teacher guidance and student autonomy, adapting to the learners’ needs and the complexity of the task.” (Dobber et al., 2017)

This is a big issue as too much agency would leave the Learner lost and confused, whereas too much guidance resolves this whole system down to rote learning.

It is imperative to have as many indicators and metrics to regulate the level of Guidance the AI Tutor offers. We already have some methods, such as the Component count within the Orientation Phase alongside the Exhaustion Fallback within the Pedagogical Rules.

Another potential method would be an **Introductory Tutorial / Calibration**, requesting basic heuristic information about the Learner (what Educational Stage they are in, etc.) as well as running through an example Question to see how the Learner interacts with this type of learning in a more controlled environment.

On the hardest open problem: the *Decision Policy* (see *MVP flow below*) is the genuine intellectual core of this system and is, as yet, not truly realised. How exactly a Component's **mastery**, **groundedness** and **attempts** scores translate into a concrete action (ask Inquiry / request evidence / hint / advance) is the central design question, and it is best resolved *empirically* — by observing the Tutor in testing and tuning the thresholds — rather than fixed a priori. We name this openly as the system's hard part; it is the most substantive subject for the Task 4 reflection, where the gap between the written rules and the realised behaviour under test is the analysis itself.

Debug Mode

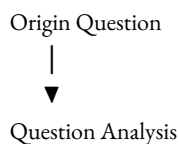
The Debug View is not future work — it is the window into the state machine, and the primary means by which the Decision Policy is observed and tuned during testing. It exposes the live state, e.g.:

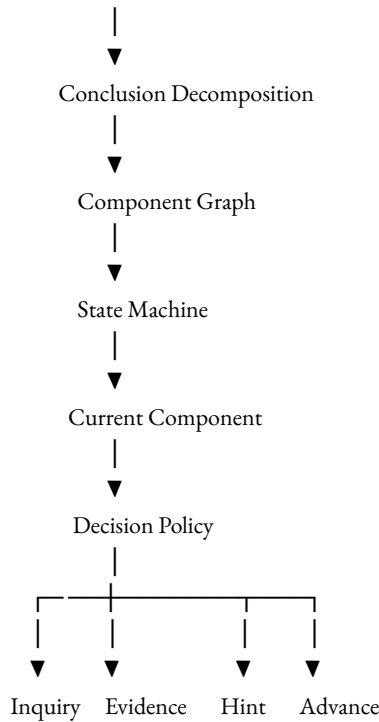
Current Component: Cooling
Confidence: 72%
Covered: No
Evidence Seen: 2
Next Candidate: Condensation
Reason: Learner correctly explained evaporation but has not justified
why cooling causes condensation.

And the underlying state:

```
{  
  "component_mastery": {  
    "evaporation": 0.8,  
    "condensation": 0.3,  
    "cooling": 0.6  
  },  
  "evidence_quality": 0.7,  
  "attempts": 2,  
  "guidance_level": 1  
}
```

System Flow





The branch point is the **Decision Policy** — the box that, given the current Component’s **mastery**, **groundedness** and **attempts**, selects the next action. As noted above, this is the system’s intellectual core and the subject of empirical tuning.

MVP & Build Order

The full architecture above is the **target system (v1)** — *not* the minimum to build first. Attempting to build every subsystem before anything runs is the principal risk given the timeframe; a realistic target is implementing 60–70% of what is specified here, with the layered build below ensuring that whatever is finished, runs. Each layer runs end-to-end before the next begins:

Layer 1 — the demoable core (build first). A Gradio chat connected to the LLM with a single system prompt running a three-Phase loop (ask → investigate → conclude) that never reveals the answer before a Hypothesis is recorded. The **target_conclusion** may come from the model’s own knowledge at this stage; minimal state (current Phase, Hypothesis recorded). This alone is a complete, demoable inquiry tutor.

Layer 2 — what makes it defensible. Add Wikipedia grounding (answer key + evidence from retrieved text), and replace in-prompt heuristics with the real structured pipeline: Question Analysis → answer-key generation → Conclusion Decomposition producing **required_components**, with mastery/groundedness tracked in *code*.

Layer 3 — deliverable & polish. The Recursive Audit (post-hoc), then the Exhaustion Fallback ladder, then — only if the above is solid — the inquiry-graph render. The richest portion of remaining time is then spent tuning the Decision Policy, which only becomes testable once Layers 1–2 give it a substrate to run inside.

